

The Performance Commons

Why ASIC-level performance is a collective-action problem, and the hardware we should build for its solution

A position paper. The figure and the underlying architecture analysis are drawn from the companion technical comparison, "Dataflow vs. an Optimal Heterogeneous Architecture."

Abstract

The reason we don't achieve near ASIC performance and power (and the reason we don't have much better ASICs) for many more applications is not technical in nature. It is the collective action problem both within and across levels of the computing stack. As machine intelligence continues to advance, as long as it maintains the apparent inoculation to this problem, this fact will increasingly be exposed. This raises many questions for forward looking engineers in the form of, "How do we design systems assuming the collective action problem is solved?". For example, how do we design hardware assuming there will be orders of magnitude more intelligence available for shared code optimization? We likely want more reconfigurable and less "programmable" architectures. And similarly, how do we design software stacks assuming the programmability issues holding back wider adoption of distributed computing are solved?

1. The gap is real, and it is not physics

A general-purpose core is roughly **100–500× less energy-efficient than an ASIC** for the same task (Hameed et al., ISCA 2010). Word-level reconfigurable hardware (a CGRA) sits within $\sim 2.8\times$ of an ASIC; bit-level reconfigurable hardware (an FPGA) $\sim 20\text{--}35\times$ off (Prabhakar et al., ISCA 2017; Kuon & Rose, 2007). Most general-purpose code runs one to two orders of magnitude below the achievable frontier.

The companion analysis shows this gap is not bound by physics. A perfect compiler and scheduler brings each substrate to its roofline, and the classic limit studies (Lam & Wilson, ISCA 1992; Wall, ASPLOS 1991) locate the dominant barrier in *control flow and the effort of exposing parallelism* — not in raw data dependence. The headroom is real and substrate-reachable. What separates achieved from achievable is therefore, overwhelmingly, the **effort of specialization** — writing the optimal kernel, building the accelerator, tuning to the platform — and not a wall in the silicon.

Substrate	Penalty vs an ASIC (same function)	Measured by
ASIC	1× — the efficiency ceiling	—
CGRA (word-level reconfigurable)	$\sim 2.8\times$ area	Prabhakar et al. (Plasticine), ISCA 2017
FPGA (bit-level reconfigurable)	$\sim 20\text{--}35\times$ area, $\sim 3\text{--}4\times$ delay, $\sim 10\text{--}14\times$ power	Kuon & Rose, 2007
General-purpose CPU	$\sim 100\text{--}500\times$ energy	Hameed et al., ISCA 2010

Table 1. The measured cost of generality. These are the intrinsic, physical penalties — the realizable frontier of Figure 1 — not what most code actually achieves, which is typically worse because the substrate's potential goes unextracted.

2. The general-purpose vs. custom trade-off is mostly an effort artifact

The trade-off at the center of computer architecture is usually drawn as a Pareto frontier between **flexibility** and **efficiency**: a CPU does anything but wastes energy; an ASIC wastes nothing but does one thing; FPGAs and CGRAs sit in between. The conventional reading is that you *buy* efficiency with flexibility and pick your point on the curve. But that curve is drawn at a fixed, finite *programmer effort* — it is a slice through a three-dimensional surface (flexibility $\times$ efficiency $\times$ effort), and the slice moves as effort changes.

To see why, decompose the efficiency gap between a reconfigurable substrate and an ASIC into two parts. The first is the **intrinsic fabric tax**: the physical overhead of the reconfigurable silicon itself — the switches, the configuration memory, the over-general datapath. This is irreducible; no amount of programmer effort removes it, and it is what the measured ratios capture (a CGRA $\sim 2.8\times$ an ASIC's area; an FPGA $\sim 20\text{--}35\times$). The second is the **realization gap**: how much of the substrate's *available* efficiency you actually extract, which depends entirely on the quality of mapping, placement, and scheduling — and therefore on effort. At finite effort you extract a fraction; at the limit (the oracle mapping of the companion report) you extract all of it.

The consequence is the whole argument in one picture. As effort rises, the realization gap closes and the achieved-efficiency curve falls toward the realizable frontier — which is just the intrinsic fabric tax. Crucially, **the effort gap is concentrated exactly on the reconfigurable substrates**: it is $\sim$ zero at the custom end (an ASIC has nothing to program) and small at the general-purpose end (CPUs are easy to target and their compilers are mature), but it is enormous in the middle, where reconfigurable hardware lives. This is precisely why reconfigurable fabrics have historically underdelivered — not because they are inefficient, but because their efficiency was unreachable at affordable effort. Close that gap — which is what a shared, AI-driven optimization commons does — and a CGRA sits within $\sim 2.8\times$ of an ASIC *while keeping full flexibility*. The flexibility–efficiency trade-off does not so much shift as **flatten**: under unlimited effort, the only efficiency you still surrender for flexibility is the small, physical fabric tax.

The general-purpose vs. custom trade-off is mostly an effort artifact

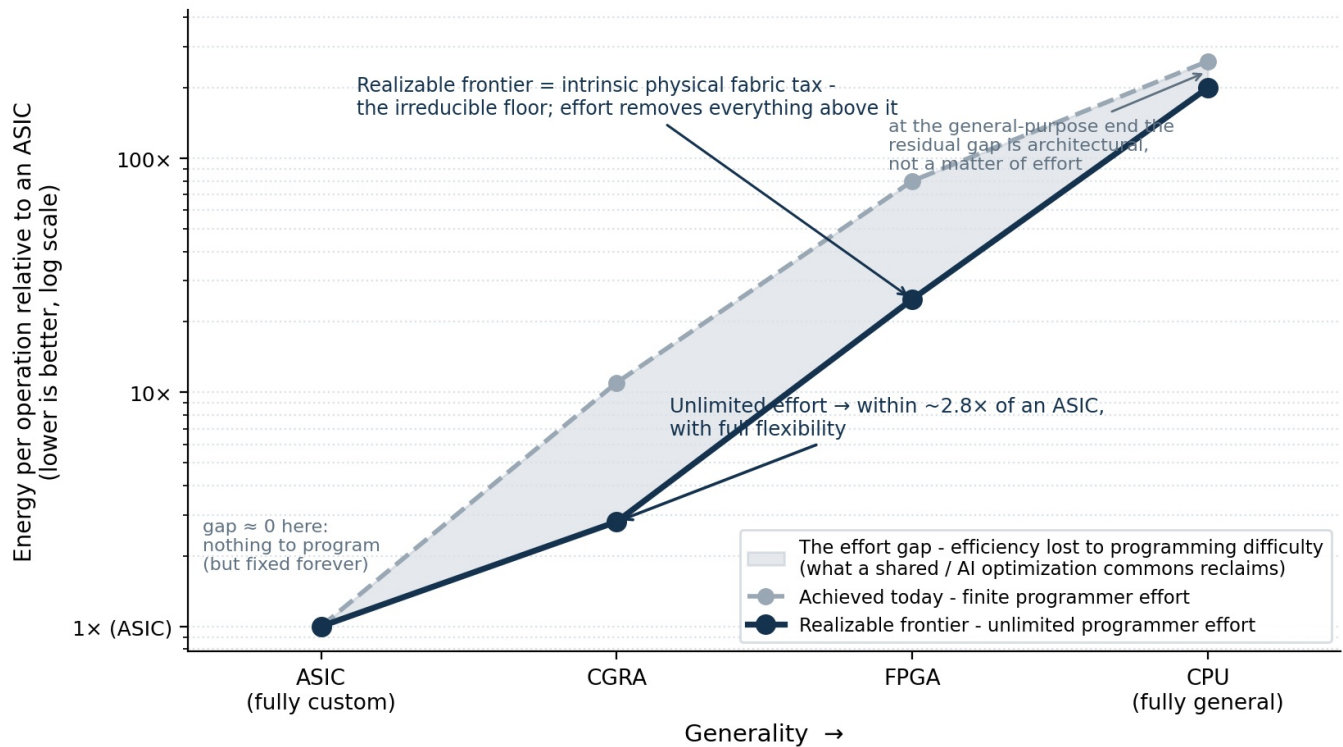


Figure 1. The canonical flexibility–efficiency trade-off is largely an artifact of finite programmer effort. The shaded band is the effort gap — efficiency forfeited because extracting a reconfigurable substrate's potential is too laborious to afford per workload. It is widest exactly where reconfigurable hardware sits, ~zero at the custom end (nothing to program), and architectural rather than effort-bound at the general-purpose end. A shared / AI optimization commons reclaims the band; what remains is the intrinsic physical fabric tax (CGRA ~2.8×, FPGA ~20–35×). Substrate-to-ASIC ratios on the realizable frontier are anchored to measured values; the finite-effort curve and the effort axis are illustrative.

One more turn of the screw closes the loop with the design conclusion. Even *unlimited design effort* — machine intelligence spinning up bespoke ASICs cheaply — does not argue for ASICs everywhere, because an ASIC is fixed at tape-out and real workloads are not: the adaptability an ASIC structurally lacks is exactly what its efficiency edge over a CGRA costs. So unlimited effort, applied honestly, does not push the optimum toward *more custom* silicon; it pushes it toward **reconfigurable** silicon that an oracle-grade commons can drive to within a small factor of custom while remaining able to change what it computes. That is the analytical core of "more reconfigurable, less programmable."

3. Near-oracular performance is a privilege

Consider who actually reaches the frontier today: hyperscalers with custom silicon, high-frequency-trading firms, AAA game-engine teams, codec and DSP vendors, and the handful of groups who hand-write CUDA or assembly. They get there through enormous, bespoke, and largely unshared effort, justified only by scale or stakes. Everyone else ships code far off the frontier — not for lack of talent or physics, but because the cost of specialization is affordable only at those scales. Performance is gated by *who can pay the specialization cost*, not by what is achievable.

4. The privilege is a collective-action problem

An optimization is a **non-rival good**: my hand-tuned FFT does not diminish yours. Classic economics says non-rival goods are under-provided whenever no single actor captures enough of the aggregate benefit to justify producing them for the whole population. So each team re-derives or hoards, and the population sits below the frontier. This is the textbook structure of a collective-action problem — under-provision of a public good. The performance gap is not a technology gap; it is a coordination failure.

5. The same logic explains the absence of better ASICs

One level down, ASIC *design and optimization* knowledge — mapping heuristics, verified IP blocks, the place-and-route and microarchitectural tricks that separate a good datapath from a mediocre one — is equally non-rival and equally hoarded. EDA vendors are a partial, proprietary aggregation point; the rest is re-derived bespoke per chip and locked inside firms. So the field produces fewer and worse ASICs than it collectively could, for the same reason: the design effort is not pooled.

This is the aggressive claim, so state the caveat plainly: fab economics and the genuine bespokeness of different computations are also real costs, and pooling cannot erase them. The claim is *not* that collective action is the only barrier — it is that the **pooling-addressable fraction** of the gap is large and currently wasted.

Layer of the stack	The hoarded / secret non-rival good	Open-commons counter-trend
Foundry / process	PDKs & device models under NDA	SkyWater & IHP open 130 nm PDKs
CAD / EDA tooling	Synthesis, place-&-route, timing heuristics (Synopsys/Cadence/Siemens)	OpenROAD, OpenLane, Yosys
Hardware IP blocks	Verified IP licensed proprietary	OpenCores, CHIPS Alliance, OpenHW CORE-V; Rocket/BOOM/CVA6
ISA	Proprietary ISAs (x86, ARM)	RISC-V
Microcode / firmware	Encrypted microcode, proprietary blobs	coreboot/LinuxBoot, OpenBMC, OpenTitan
Math / ML kernels	Fastest per-chip kernels hoarded (MKL, cuBLAS, cuDNN)	OpenBLAS, BLIS; TVM/Halide/Triton/Exo
GPU drivers / runtime	Per-application driver optimizations as a moat	Mesa, NVK
Workload / measurement data	Real traces siloed in the firms that own the fleets	Google cluster traces, MLPerf
Distributed compute	Idle compute unharnessed — hard to program <i>and</i> no incentive to pool	BOINC/Folding@home; DiLoCo/Hivemind/Petals

Table 2. One pattern, every layer. At each level a non-rival good — design knowledge, optimization, measurement — is hoarded or secret, and at each level an open commons is forming to route around it. The top five rows are really a single problem: the hardware-design commons that does not yet exist, from secrecy in the foundry's PDKs up through CAD tooling and IP to the ISA itself.

6. Open source already solved this — and did it by itself

The compiler and library layer is a *solved instance* of exactly this problem. LLVM, GCC, the Linux kernel, and BLAS/LAPACK are pooled optimization commons: every program compiled with LLVM inherits the accumulated optimization effort of thousands of contributors it never paid for and could never have produced alone. Crucially, this commons formed **organically**, because the cost of contributing to a shared optimizer is far below the benefit everyone draws once it exists. The trajectory is established. The only open question is how far up the stack it extends — from instruction selection (already done) into algorithm selection, runtime specialization, and ultimately hardware mapping.

Open commons	Out-competed	Fitness evidence	Layer
Linux	Proprietary UNIXes (Solaris, AIX, IRIX)	100% of the TOP500; dominates cloud & servers; Android	OS
LLVM	Proprietary / fragmented compiler back-ends	Substrate for Clang, Swift, Rust, Julia; industry-wide	Compiler
RISC-V	Proprietary ISAs (x86, ARM)	13B+ cores by end-2023; Qualcomm 650M+, NVIDIA's CUDA cores on RISC-V; Esperanto's 1,088-core chip taped out without ~\$25-30M in ARM fees	ISA
Mesa	Proprietary GPU drivers	Open driver stack reaching/beating proprietary parity on Linux	Driver
PyTorch / TensorFlow	Proprietary ML toolkits	The substrate of essentially all ML research and much production	ML framework
FFmpeg	Proprietary media / codec libraries	Underlies most of the world's media processing	Codec

Table 3. The commons wins at six different layers. Each out-competed proprietary, siloed alternatives — the pattern the paper argues is now climbing toward hardware.

The reason is structural: open commons have higher fitness in the literal variation-selection-inheritance sense. More contributors generate more variation than any single firm's team; forking is selection without permission, so good branches are adopted and bad ones die with no proprietary roadmap gating exploration; zero marginal cost of replication plus network effects spread the fitter variant and accrete an ecosystem that raises switching costs; and there is no single point of strategic death — a company can be acquired, pivot, or die and take its tooling with it, whereas a commons persists and forks, accumulating improvement over many more generations. Each improvement is inherited by every future user. That is why, once viable, the commons tends to win — and it is why expecting it to keep climbing the stack is not wishful but the default. The one qualifier sharpens the prediction: the commons wins where the artifact is non-rival and modular, contribution cost is low relative to the shared benefit, and no capital or data moat dominates — which is exactly why it has already taken compilers, the OS, and the ISA, is taking drivers and kernels now, and has not yet cracked leading-edge EDA or fabs, where capital and tacit secrecy are the moat.

7. Machine intelligence multiplies the commons; it does not replace it

It is tempting to think that sufficiently capable AI makes the commons unnecessary — each team's AI simply specializes its own code. This is the wrong conclusion, for a specific reason: in the near term, reaching near-oracular performance still takes significant effort *even with machine intelligence in the loop*. AI lowers the per-instance cost of specialization; it does not drive it to zero.

So AI does not dissolve the collective-action problem — it raises the stakes of solving it. The thing being pooled becomes more powerful per unit (AI-

generated optimizations alongside human ones), which makes the commons *more* valuable, not less. **AI plus pooling compounds; AI alone, hoarded per team, simply makes a few elite groups faster and widens the gap.** Capability is not the lever. Collaboration is. This is the sense in which machine intelligence is *inoculated* against the collective-action problem: a shared or copyable model aggregates optimization knowledge across the whole population without the incentive to hoard that defeats human coordination — and as it does, it exposes how much of the performance gap was only ever a coordination failure.

8. The aggregation point already exists

A collective-action problem is solved by an actor with the **vantage**, the **channel**, and the **incentive** to internalize the externality. The hardware-and-runtime vendor layer has all three: it is the common substrate (it alone sees the whole workload distribution — the only vantage from which population-level optimization is even possible); it owns the distribution channel into the optimization layer (microcode, drivers, firmware, on-device runtimes); and it captures the aggregate benefit (real-world performance on its silicon converts directly to competitive advantage).

This is already happening in fragments. GPU vendors ship per-application optimization profiles, aggregated across all users of a title, through driver updates. Mobile runtimes ship cloud profiles that pre-optimize a fresh install from the population's hot paths. The vertically integrated vendor that owns silicon, OS, compiler, and frameworks is the existence proof of the full stack. The "processor that reads code as a contract" is the generalization: hardware that honors the *user-visible outcome* and is free to choose any legal behavior satisfying it, with the optimization policy improving across the federation.

9. The implication: design hardware for that future

This is the position. If the optimization layer is becoming a powerful, shared, increasingly intelligent commons — and open source plus the vendor-aggregation incentive say it is — then the dominant design error is to keep building hardware optimized for the *isolated-elite-team* world. Hardware should be designed assuming the compiler and runtime it will actually have: near-oracular and collaborative. Two specifics follow.

(1) Trade programmability for reconfigurability. This is the direct consequence of §2 and Figure 1. The flexible architectures of the past — dataflow machines, CGRAs, VLIW — lost not on efficiency but on *programmability*, and that programmability tax is exactly the effort gap a near-oracular, shared optimization layer reclaims. Once the commons can target a hard-to-program fabric well, paying the small intrinsic reconfigurability tax (a CGRA's $\sim 2.8\times$) to keep full flexibility becomes the *better* deal — because the commons, not each team, pays the programming cost, once, for everyone. Build for the compiler you will have, not the one you had. Concretely, this means architectures that expose a **wide legal-behavior set** — reconfigurable, specializable, heterogeneous — with first-class hooks for contract-conditional optimization (algorithm substitution, approximation control, dynamic re-placement) and the verification and quality-monitoring those require, so that inference proposes and verification disposes.

(2) Make the optimization loop run on the federation it optimizes. The commons should not be merely a central database of optimizations pushed down to passive devices. The optimization *search itself* — and the ML models that drive it — should run across the distributed consumer-hardware federation. This is not speculative. Volunteer computing (Folding@home, BOINC) showed decades ago that idle consumer cycles aggregate into supercomputer-scale resources, and recent work shows even frontier-scale ML training survives the move off the datacenter: DiLoCo (Google DeepMind, 2023) trains language models across poorly connected islands of devices while communicating $\sim 500\times$ less than synchronous training; OpenDiLoCo replicated it across two continents and three countries at 90–95% compute utilization, with devices joining and leaving mid-run; DiLoCoX scaled the approach to a 107-billion-parameter model; and Hivemind and Petals run training and inference of very large models over thousands of volunteer nodes. The common lesson is that consumer-hardware federations are **poorly connected, heterogeneous, intermittent, and unreliable**, so they demand ML architectures and optimization loops *designed for that regime* — low-communication, fault-tolerant, asynchronous, peer-to-peer — rather than the tightly-coupled datacenter assumptions baked into today's methods. Hardware should be designed to **participate**: to contribute spare cycles to the shared optimization loop as a first-class function, making every device simultaneously a beneficiary and a contributor. That is what closes the loop — the commons is powered by the very hardware it improves. And the barrier here is the same shape as the one in Figure 1: not physics, but programmability and coordination (Figure 2).

Distributed computing is a collective-action problem, not a hardware limit

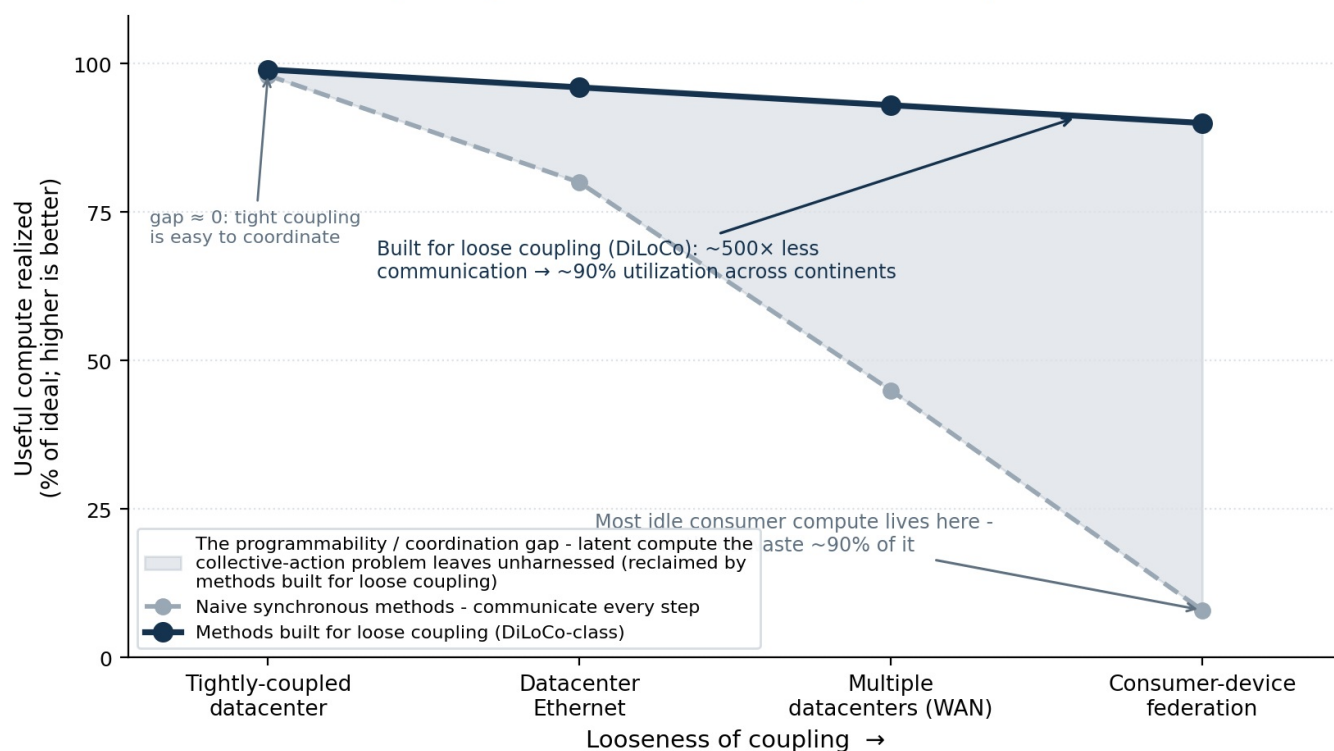


Figure 2. The same shape of problem, one level up. The aggregate idle compute in consumer devices is latent not for hardware reasons but because coordinating loosely-coupled, heterogeneous, intermittent hardware is hard to program — and because no individual is incentivized to pool it. Naive (communicate-every-step) methods waste almost all of it at the loosely-coupled end; methods built for loose coupling (DiLoCo and successors: $\sim 500\times$ less communication, $\sim 90\%$ utilization across continents) reclaim it. As in Figure 1, the gap is concentrated where the resource is hardest to reach, and it is an artifact of programmability and coordination, not physics. Utilization figures are illustrative, anchored to reported DiLoCo / OpenDiLoCo results.

Two visuals close the argument. The first asks *where the optimum lands* as the commons matures: with the programmability tax falling over time, the design point for the large, high-diversity space of real workloads migrates out of general-purpose silicon and into reconfigurable fabrics — not into fixed ASICs, which workload diversity rules out (Figure 3). The second shows this is not merely a projection from first principles. Reconfigurable hardware has been entering the datacenter for a decade: Microsoft put FPGAs in Azure servers, Intel paid \$16.7B for Altera (2015) and AMD \$49B for Xilinx (2022, the largest semiconductor acquisition ever), and the current AI-accelerator wave is largely dataflow and reconfigurable. Adoption has been gated precisely by programmability — the same barrier — which is why a shared, AI-driven commons that finally closes it is the plausible inflection, not another decade of slow entry (Figure 4).

Where the optimum lands: the reconfigurable region expands as the commons matures

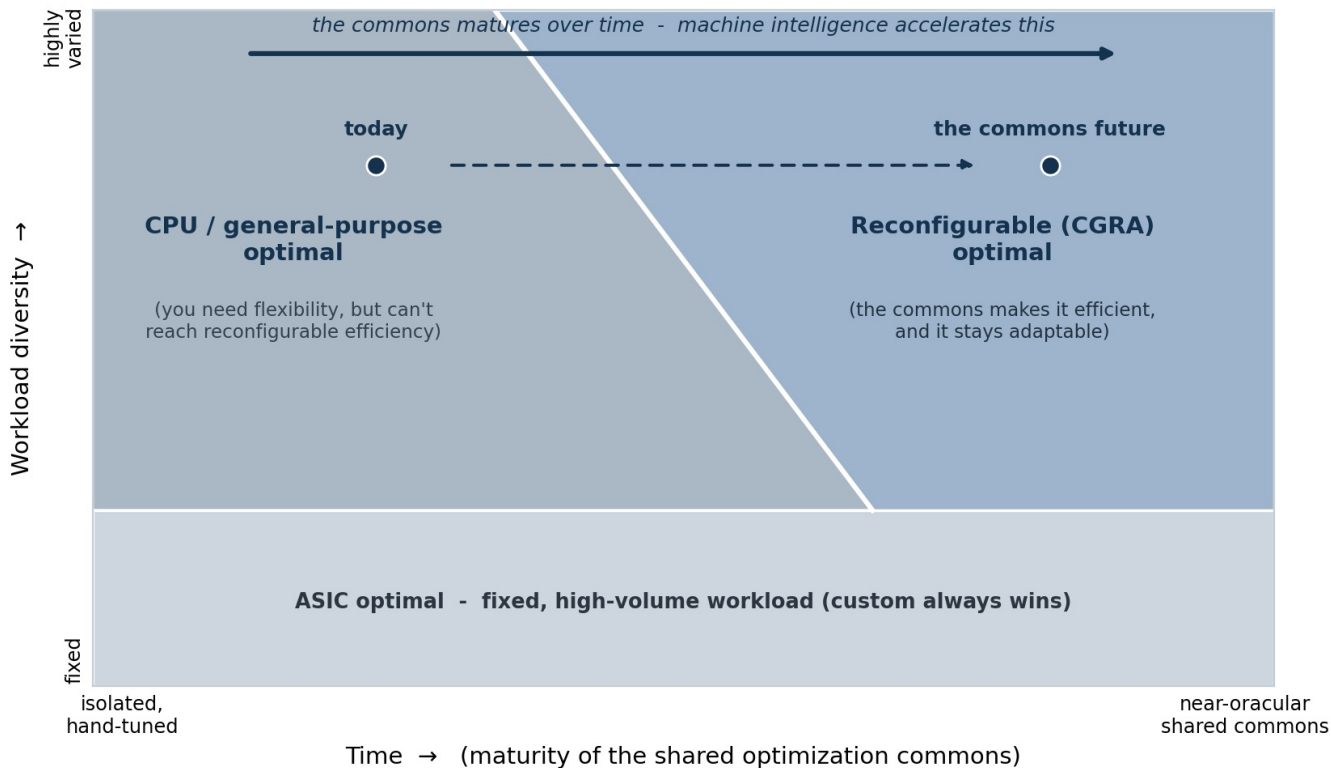


Figure 3. Where the optimum lands. Over the two axes that actually decide the choice — the maturity of the shared optimization commons (which grows over time, accelerated by machine intelligence) and workload diversity — fixed ASICs remain optimal only for stable, high-volume workloads; the large high-diversity space currently defaults to general-purpose silicon because the commons is weak; and as the commons matures the reconfigurable-optimal region sweeps left into that territory. The boundary is conceptual — pinning down its real position is exactly what the whole-system characterization of \$10 would enable.

Reconfigurable hardware is already moving in — the commons is the inflection

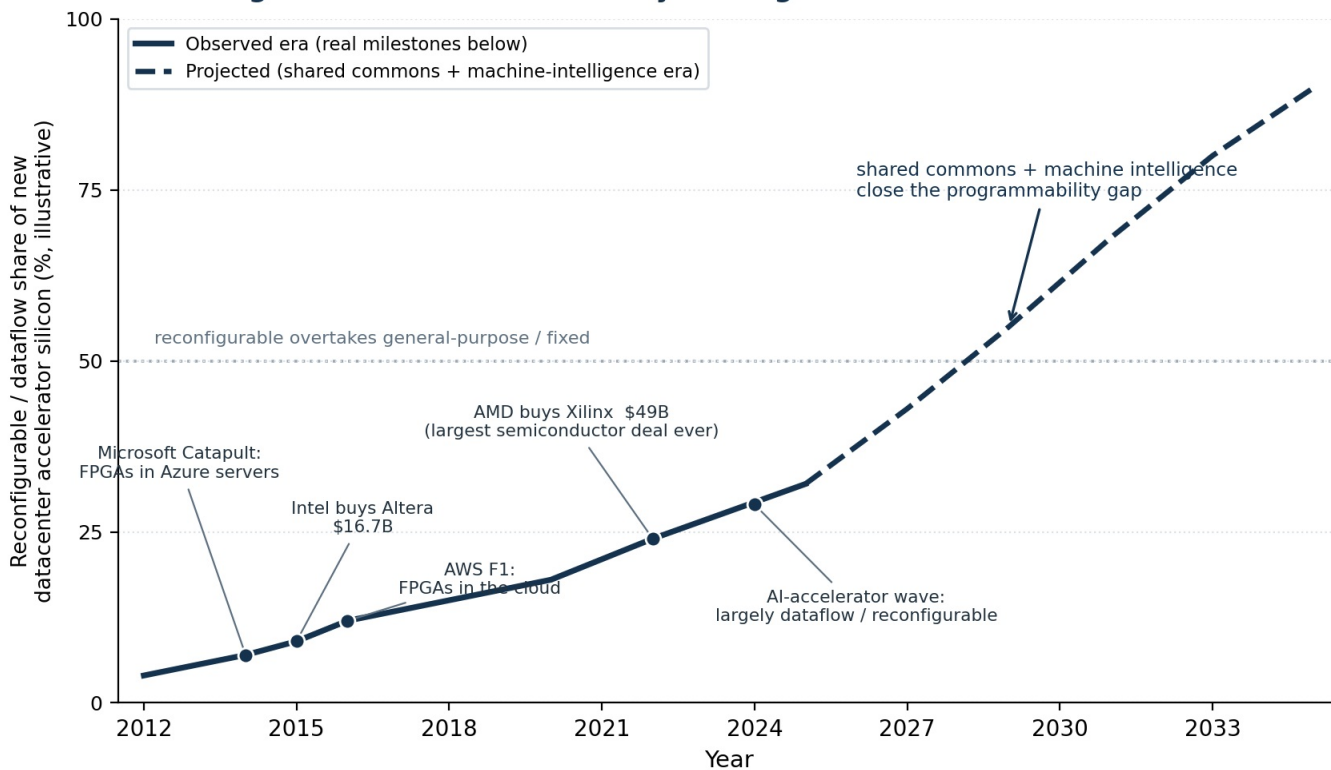


Figure 4. The migration is already underway. The dated milestones and acquisition values are real (Microsoft Catapult; Intel–Altera \$16.7B, 2015; AWS F1; AMD–Xilinx \$49B, 2022); the share curve is an illustrative trend and the post-2025 segment a projection. Reconfigurable hardware’s datacenter adoption has been gated by programmability — Intel later sold down Altera as the broad FPGA thesis underdelivered for exactly that reason — which is why closing the programmability gap, via a shared and AI-driven optimization commons, is the plausible inflection.

10. Honest limits

- **Physics still floors it.** Pooling exploits the legal slack the physics leaves; it cannot beat data-dependence height or genuine serialization. The commons widens what is reachable; it does not repeal Amdahl.
- **Whoever runs the loop owns the objective.** Solving the collective-action problem by *concentrating* it trades it for a principal-agent problem: the aggregator optimizes *its* metric, which may not be the user’s contract, and the commons can wall per vendor and entrench incumbents through a data-network-effect flywheel. The design mandate therefore includes building for **open, user-verifiable** commons. The difference between a commons and a moat is governance — and that choice should be made in the architecture, not left to the incumbent.
- **We are still missing the map.** Targeting this well requires a whole-system characterization of real consumer workloads — maximum-versus-achieved parallelism, dataflow locality, phase and input statistics — that does not yet exist as a single study. That measurement is the prerequisite, and building it is the first concrete step.

Conclusion

The distance between the performance we get and the performance the silicon allows is, to first order, a coordination failure, not a physics one. We solved the same failure once already, at the compiler layer, with an open optimization commons that formed on its own and now hands near-expert performance to everyone for free. Machine intelligence will make that commons far more powerful — but only collaboration turns capability into *universal* performance rather than a widening gap between the few and the rest. The strategic move is therefore not to build faster rigid chips for a world of isolated optimizers, but to build **collaboration-native hardware** for the world that is arriving: reconfigurable rather than merely programmable, instrumented for contract-conditional optimization, able to contribute to a distributed optimization loop running on consumer hardware, and governed as an open commons rather than a private moat. The commons is coming for hardware the way it came for software. We should design the hardware to meet it.